PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Project Overview 3
2. Background & Business Objective 3
3. Product Purpose 4
4. System Functionalities 4
5. User Stories & Use Cases 7
6. Features Included (Scope Definition) 9
7. Features Not Included (Scope Control) 9
8. Assumptions & Constraints 9
9. Risks & Questions Throughout Development 10

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: ZenTravel

Version: 1.0

1. Project Overview

- **Problem Statement:** When flights are delayed, cancelled, or baggage is lost, travellers must piece together rebooking, accommodation, and compensation steps across airline policies, insurers, and ad-hoc search. That work is time-consuming, jargon-heavy, and error-prone especially under stress. There is no single place that turns messy evidence (messages, screenshots, PDFs) into a structured, actionable recovery path.
- **Target Domain:** Agentic Workflow Automation for Travel Incident Management and Resolution - AI-assisted travel disruption recovery and trip planning, delivered as a cross-platform travel companion, with Firebase-backed identity and booking data and GLM-powered reasoning.
- **Proposed Solution Summary:** ZenTravel combines a Brain Master multi-stage workflow (extract → validate → ReAct plan → tool execution → synthesis) with specialist agents (flight, hotel, insurance, trip, car), document ingestion (PDF text, OCR for images, plain text), and mock travel APIs that simulate sourcing and compensation checks. The product shifts the user from hunting scattered information to reviewing and approving a recovery plan, with optional email delivery (EmailJS) and PDF itinerary export where implemented.

2. Background & Business Objective

- **Background of the Problem:** Traditional recovery is linear and manual: the traveller reads notices, calls carriers, searches for hotels, and drafts emails without a shared model of "what happened" or "what to do next." Product and engineering demos often separate

© UMHackathon 2026

"chat" from "booking data" and from evidence (boarding passes, SMS). ZenTravel is designed to unify narrative input, document signals, and structured planning in one app shell.

- **Importance of Solving This Issue:** By automating disruption workflows, ZenTravel reduces resolution time from hours of stressful phone calls to minutes of AI-driven orchestration. It shifts the traveler's role from a frantic "coordinator" to a calm "approver" of AI-proposed solutions.
- **Strategic Fit / Impact:** This project perfectly demonstrates the capability of Z.AI GLM to act as a central reasoning engine (via ILMU OpenAI-compatible API, model ilmu-glm-5.1). It showcases multi-step decision-making, API interaction, and the handling of highly unstructured, real-world ambiguity, proving that AI can execute critical operational workflows beyond simple chatbot interactions. It uses Firebase for auth and user/booking persistence, and Capacitor (com.zentravel.app) to package the React app for Android distribution using the built dist bundle.

3. Product Purpose

3.1. Main Goal of the System: To provide an autonomous, stateful agent that detects travel disruptions and automatically executes recovery workflows, presents structured plans with human approval, alongside core app features (bookings, savings, notifications, trip planning).

3.2. Intended Users (Target Audience):

- Frequent Business & Leisure Travelers
- Travel Insurance Providers (B2B integration potential)
- Airline Customer Service Teams (for automated triage)

4. System Functionalities

4.1. Description: ZenTravel operates as a reactive, stateful workflow engine. Once triggered by a disruption event, the Z.AI GLM parses the intent, formulates a multi-step recovery plan, executes API calls (mocked for the hackathon) to external services, and maintains the context of the user's situation until the crisis is resolved.

4.2. Key Functionalities:

- **Brain Master workflow:** Five visible stages (Input Agent → Validation → Reason → Act → Observe) plus a strategy step; uses GLM for JSON extraction, tool planning, observation/synthesis, and recovery strategy when available; falls back to regex / templates when GLM fails or is unconfigured.
- **Specialist agents:** Single-shot GLM calls with strict JSON shapes for domain prompts (flight, hotel, insurance, trip, car).
- **Document & image intake:** Upload on Brain Master path; PDF text via pdf.js, images via Tesseract.js OCR, capped raw text for downstream parsing; extracted text feeds Stage 1 (no separate "pre-summary" GLM call in the current pipeline to save cost).
- **Trip planning:** Destination templates, mock offers, optional GLM narrative, PDF itinerary download.
- **User account & data: Firebase Auth;** Firestore users, Booking, search_history; profile preferences and cashback display.
- **Operational app features:** Google sign-in, Firestore sync (currency, language, cashback), Booking collection live listener, search_history, Saved items, My Reviews, Refund flow, Help Center, About, Edit Profile, category hubs (Hotels, Flights, Insurance, Car Rental), notifications, exchange-rate helpers.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: Z.AI GLM is utilized as the central orchestrator. Z.AI GLM is exposed through the ILMU OpenAI-compatible API — default ilmu-glm-5.1 (configurable via

© UMHackathon 2026

VITE_GLM_MODEL), with base URL from VITE_GLM_API_URL or dev proxy. The product requires structured JSON extraction, multi-step reasoning, and tool planning over travel-specific vocabulary. GLM 5.1 is used as a thinking model: the client accounts for reasoning_content vs visible content, uses a minimum max_tokens floor, and can expand the token budget on finish_reason: length retries. Temperature 0 reduces exploratory drift for compliance-style outputs (claims, emails).

4.3.2. Prompting Strategy: Multi-step, staged prompting with ReAct-style separation (Reason → Act → Observe) rather than one monolithic completion:

- **Stage 1:** System prompt constrains output to a fixed ExtractedIncident JSON schema; user message is natural language (and may include OCR noise).
- **Stages 3–5:** GLM proposes a **tool plan**; the app **executes tools deterministically**; a further GLM pass **observes** real tool JSON and produces narrative + ranked options.
- **Specialized agents:** Single request per turn with **strict JSON output contract** and clarifying-question arrays when information is missing.

4.3.3. Context & Input Handling:

- **Workflow Stage 1 user text:** input.rawInput is truncated to 8,000 characters before the GLM extraction call.
- **Vision/ document pipeline:** Extracted text passed to the workflow is capped at 4,000 characters; plain text file reads are truncated at 6,000 characters in the reader.
- **Unsupported or empty files:** Returns empty extraction with glmInterpreted: false; very short OCR (< 10 chars) yields no usable text.
- **GLM completion budget:** max_tokens starts at least 1,024 and can increase on retry up to 8,192 when the model hits length with empty visible content.
- **HTTP:** 90 s client timeout; up to 2 retries on 5xx, timeout, network errors, or empty body scenarios as implemented in GLMClient.

4.3.4. Fallback & Failure Behavior:

© UMHackathon 2026

- **GLM missing or failing (Stage 1):** Regex/ heuristic extraction (buildFallbackExtraction) supplies disruption type, flight number, and route guesses with confidence and missingFields.
- **GLM tool planning / observation:** Rule-based fallbacks in the ReAct layer (per tools.ts) reduce adaptability but keep the pipeline running.
- **Strategy step:** Defaults to a short template string if GLM output is invalid or out of length bounds.
- **Clarification:** Workflow returns stage: "paused" with clarifying questions when validation fails such as unknown disruption type, missing destination.
- **All automated actions failed:** stage: "failed" with user-facing message steering toward specialist agents.
- **Human escalation:** Explicit approval on recovery actions (userApprovalRequired: true); EmailJS / mailto for outbound communication.
- **UI:** Errors surface as console warnings and user-visible retry messaging where the client throws.

5. User Stories & Use Cases

- **User Stories:**
  - "As a stranded traveler, I want to upload a screenshot of my flight cancellation so that the system can automatically suggest a new itinerary and book a nearby hotel for the night."
  - "As a user, I want to approve each recovery action so that nothing is treated as “done” without my consent. "

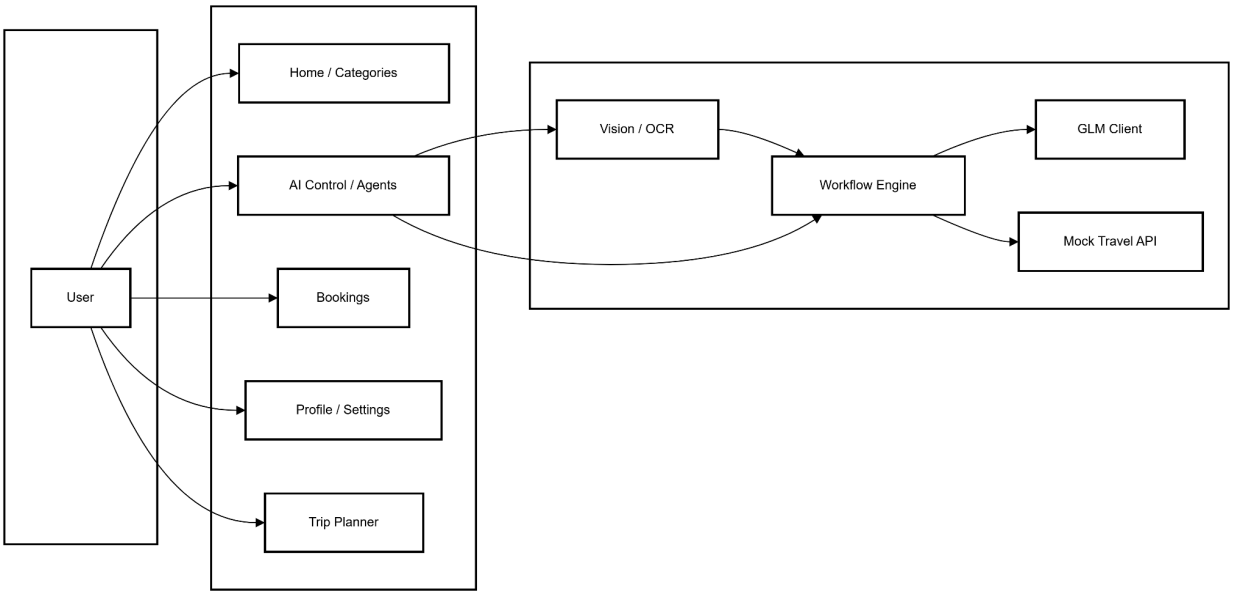
- **Use Cases (Main Interactions):**

  - **Disruption → Recovery plan (Brain Master):** User opens AI Control, selects Brain Master, enters text or attaches a document → Stage 1 extraction →

validation (optional pause for answers) → ReAct tool execution on mock APIs → synthesis → strategy → user reviews `WorkflowResponseCard` and approves actions → optional recovery email.

- **Specialist consultation:** User selects Flight / Hotel / Claims (insurance) / Trip / Car agent → single-turn or clarifying JSON response with suggested emails, drafts, or search-style actions.
- **Trip planning:** User sets dates, destination, preferences → system composes template days, mock flight/hotel, optional GLM narrative → export PDF.
- **Booking management:** User opens Bookings tab → Firestore-backed list → open ticket or refund flows linked by booking id.
- **Cross-platform use:** User installs Android app → loads bundled `dist` → same routes and AI features as web (subject to device and network).

- **Use Case Diagram:**



© UMHackathon 2026
Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- Firebase authentication (including Google) and Firestore persistence for users, bookings, and search history
- Brain Master 5-stage workflow UI with live stage indicator and agent activity panel
- Specialized agents with distinct mascots and prompt templates
- File upload (PDF, image, text types per visionService) on the Brain Master path
- Mock travel API for deterministic flights, hotels, and compensation-style outcomes
- EmailJS recovery emails and mailto fallback
- Trip Planner with PDF export; Manual Planner page
- Hotels, Flights, Insurance, Car Rental category pages; Saved, My Reviews, Refund, View Ticket
- Notifications, Help Center, About Us, Edit Profile, currency and i18n hooks
- Floating chat entry point plus full Chatbot page
- Capacitor Android packaging (com.zentravel.app, webDir: dist)

7. Features Not Included (Scope Control)

- Real payment gateway integrations (will use simulated success states).
- Live production connections to actual airline backend systems.
- Guaranteed regulatory accuracy — EU261/MY-style rules in mocks are illustrative, not legal advice
- Multi-user/family group itinerary splitting (MVP focuses on single-user workflow).

8. Assumptions & Constraints

LLM cost constraint: A full Brain Master run may invoke multiple GLM calls (extraction, planning, observation, strategy); specialized agents add further calls. Rough order of magnitude: on the order of tens of thousands of tokens per deep session if thinking tokens consume budget (highly variable by incident complexity and max_tokens).

© UMHackathon 2026

Email: umhackathon@um.edu.my

Cost-control design decisions (examples from codebase):

- Temperature 0 and bounded max_tokens with capped growth on retry.
- Skipping a separate GLM "pre-summary" in visionService. Raw OCR/PDF text flows to Stage 1 to avoid an extra round trip.
- Truncation of user and document text before model calls.
- Mock APIs keep tool outputs compact and reproducible.

Technical constraints:

- Requires VITE_GLM_API_KEY (and related env vars) for full intelligence; otherwise fallbacks dominate.
- CORS / ILMU endpoint must be reachable from the client or dev proxy (/ilmu-api/v1 in development).
- PDF export uses standard fonts; emoji and some Unicode are sanitized for PDF compatibility.

Performance constraints:

- 90 s GLM timeout; heavy thinking models may still feel slow on poor networks.
- OCR runs in-browser (Tesseract), hence large images may be CPU-intensive on low-end devices.

User input:

- The system does not act on real airline or insurer systems on the user's behalf; approvals and outbound email are human-gated.
- Output quality depends on prompt clarity and document legibility.

9. Risks & Questions Throughout Development

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Hallucination & policy accuracy:** GLM may state entitlements or procedures not matching a specific ticket or jurisdiction; how do we surface confidence, disclaimers, and mock-data labels clearly in UI copy?
- **Similarity of plans across users:** Mock APIs and fixed routes may yield repetitive "best" options; is varied seeding or user-specific context needed for differentiation?
- **Document interpretation:** Poor scans, screenshots with glare, or non-English notices may break OCR or extraction; what minimum quality messaging and manual override flows should exist?

© UMHackathon 2026

Email: umhackathon@um.edu.my